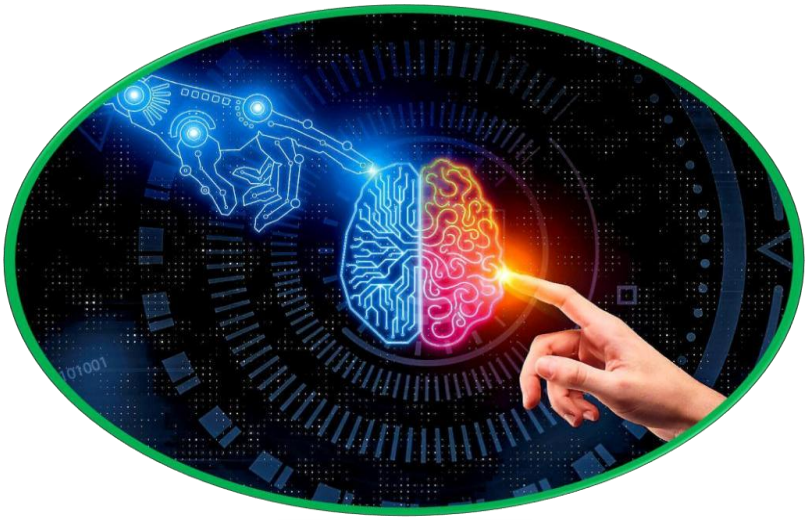
UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026

INFORME CLASE N° 3

CLASE PRODUCTO Y COMPETENCIA FINAL

CONSOLIDACIÓN DE MÉTODOS, PARÁMETROS, VOID VS. RETURN Y GAMIFICACIÓN



Integrantes:
Esteban Zambrano
Kevin Garcés
Cristian Gómez
Erick Cordonez
Docente: Ing. José Ruben Caiza Caizabuano

1. Introducción

La programación orientada a objetos (POO) organiza el código modelando objetos del mundo real. Cada objeto tiene datos que lo describen (atributos) y comportamientos que puede realizar (métodos). En Java, esos comportamientos se programan mediante métodos, y la decisión de que un método devuelva o no un valor (void o return) y de qué información necesita recibir (parámetros) determina cómo se comunica con el resto del programa.

El presente informe resume la clase N°03, titulada «Práctica evaluativa: Clase Producto y competencia final», correspondiente al cierre del Grupo 2. En él se describen, en el mismo orden de las diapositivas, la agenda de la sesión, los fundamentos clave repasados, el ejercicio en clase «Completar código» con la clase Producto, su solución, la prueba desde la clase Main, el repaso previo al juego y los ejercicios APE planteados.

La sesión tuvo como propósito evaluar y consolidar todo lo aprendido durante la semana sobre métodos, parámetros, la diferencia entre void y return y el paso por valor, combinando un taller en parejas, una socialización guiada y una evaluación formativa gamificada.

2. Objetivos

2.1. Objetivo general

Consolidar el manejo de métodos, parámetros y valores de retorno en Java mediante la resolución del taller «Completar código» de la clase Producto, la socialización de su solución y una evaluación formativa interactiva.

2.2. Objetivos específicos

- Distinguir cuándo un método debe declararse como void y cuándo debe retornar un tipo de dato (double, int, String).
- Explicar el rol de la palabra reservada return y su efecto sobre la ejecución del método.
- Completar la firma de un método, su tipo de retorno y la palabra reservada correspondiente en el esqueleto de la clase Producto.
- Instanciar la clase Producto y probar sus métodos desde la clase principal (Main).
- Diferenciar parámetros de argumentos y reafirmar que Java siempre realiza el paso por valor.
- Aplicar lo aprendido en los ejercicios APE de tipos de retorno, métodos void, múltiples parámetros y validación de datos.

3. Desarrollo

3.1. Presentación del tema y agenda de la sesión

Se presentó la clase N°03, «Práctica evaluativa: Clase Producto y competencia final», bajo el título «Consolidación de métodos, parámetros, void vs. return y gamificación». El objetivo de la sesión fue evaluar y consolidar todo lo aprendido durante la semana. La agenda se desarrolló en cuatro puntos:

N°	Actividad	Descripción
1	Dinámica en parejas	Taller «Completar código» (Clase Producto).
2	Socialización	Solución guiada y aclaración de dudas.
3	Evaluación formativa	Competencia interactiva (Kahoot / Quizizz).
4	Cierre de serie	Premiación con puntos extra y conclusiones finales.

3.2. Repaso: fundamentos clave

Antes del taller se repasaron tres ideas fundamentales sobre el diseño de métodos.

¿Cuándo usar void vs. un tipo de dato (double, int, String)? Se usa void cuando el método realiza una tarea directa hacia afuera o en el sistema (imprimir en consola, guardar un archivo, cambiar el color de una pantalla) y ningún otro bloque de código necesita usar el resultado. Se usa un tipo de dato especificado cuando el método realiza un cálculo o procesa datos y el programa principal necesita esa respuesta para guardarla en una variable, hacer operaciones subsiguientes o tomar decisiones en un if.

El rol del return: no solo envía el dato de vuelta, sino que detiene inmediatamente la ejecución del método. Cualquier línea de código escrita después de un return sin condición es inalcanzable para Java.

La regla de los parámetros: son los «canales de entrada» del método. Si el método necesita información externa que no conoce previamente (como el porcentaje de descuento de un producto), debe exigirla como parámetro dentro de sus paréntesis.

3.3. Ejercicio en clase: completar código (Clase Producto)

Se planteó una dinámica en parejas. La instrucción fue analizar un esqueleto de código e identificar los espacios faltantes (___); el objetivo, completar la firma del método, el tipo de retorno o la palabra reservada según correspondiera. El esqueleto presentado fue el siguiente:

Esqueleto de la clase Producto

```
public class Producto {
    // Atributos
    String nombre;
    double precio;

    // 1. Método para mostrar datos (¿Lleva return o void?)
    public ____ mostrarDetalles() {
        System.out.println("Producto: " + nombre + " | Precio: $" + precio);
    }

    // 2. Método para calcular el precio final con descuento (Recibe descuento en %)
    public ____ calcularPrecioFinal(double porcentajeDescuento) {
        double descuento = precio * (porcentajeDescuento / 100);
```

```
        ____ precio - descuento; // ¿Qué palabra devuelve el valor?
    }
}
```

3.4. Solución y explicación del código

En la socialización se explicaron al grupo los siguientes puntos:

- **mostrarDetalles():** usa void porque solo imprime un mensaje en consola y no devuelve ningún dato.
- **calcularPrecioFinal(...):** retorna un double porque entrega el precio con el descuento ya aplicado.
- **return:** es indispensable para extraer la operación matemática y entregarla a la clase Main.

3.5. Código resuelto

Clase Producto completa

```
public class Producto {
    String nombre;
    double precio;

    public void mostrarDetalles() {
        System.out.println("Producto: " + nombre + " | Precio: $" + precio);
    }

    public double calcularPrecioFinal(double porcentajeDescuento) {
        double descuento = precio * (porcentajeDescuento / 100);
        return precio - descuento;
    }
}
```

Espacio	Respuesta	Justificación
1	void	mostrarDetalles() solo realiza una acción (imprimir); ningún otro bloque necesita un resultado.
2	double	calcularPrecioFinal() procesa datos y debe entregar el precio con descuento a quien lo llama.
3	return	Es la palabra reservada que devuelve el valor calculado y finaliza el método.

3.6. Prueba en la clase principal (Main)

Para comprobar la clase se instanció un objeto Producto en el método main, se le asignaron valores a sus atributos y se invocaron ambos métodos.

Cómo se genera (se instancia y se invoca)

```
public class Main {
   public static void main(String[] args) {
      // Instanciamos el objeto Producto
      Producto p1 = new Producto();
      p1.nombre = "Laptop";
      p1.precio = 800.0;

      // Llamada al método void
      p1.mostrarDetalles();

      // Llamada al método con retorno y argumentos
      double precioOferta = p1.calcularPrecioFinal(15.0); // 15% descuento
      System.out.println("Precio final con oferta: $" + precioOferta);
   }
}
```

Se crea el objeto p1, se llama al método void mostrarDetalles() y se invoca calcularPrecioFinal con el argumento 15.0; el valor devuelto se captura en la variable precioOferta, que luego se imprime.

3.7. Repaso: checklist rápido antes del juego

Antes de la competencia interactiva se repasó la siguiente lista de verificación:

- **Firma del método:** define la visibilidad, el tipo de retorno, el nombre y los parámetros.
- **void vs return:** void ejecuta acciones; return calcula y devuelve valores.
- **Parámetro vs argumento:** el parámetro es el molde (double porcentajeDescuento); el argumento es el dato real (15.0).
- **Paso por valor:** Java siempre envía copias de las variables a los métodos.

3.8. Ejercicios APE

Se plantearon ocho ejercicios de práctica. A continuación se presenta cada enunciado con su desarrollo.

Ejercicio 1: Identificación de tipos de retorno. Enunciado: dado un método, explicar por qué genera un error de compilación en Java y cómo se corregiría. En Java, la firma de un método es un contrato: si declara un tipo de retorno distinto de void, todas las rutas de ejecución deben terminar en un return de un valor compatible con ese tipo; si declara void, no puede retornar un valor. Los errores de compilación más comunes son, por tanto, un método con tipo de retorno que no tiene return, o un método void que intenta devolver un valor. La corrección consiste en hacer coincidir la firma con el comportamiento: agregar el return correspondiente o cambiar el tipo de retorno a void.

Ejercicio 2: Métodos void para acciones directas. Enunciado: crear en la clase Bienvenida un método que reciba un String nombre y un int edad e imprima en consola «Hola [nombre], tienes [edad] años. ¡Bienvenido al sistema!».

```
public class Bienvenida {
   public void saludar(String nombre, int edad) {
      System.out.println("Hola " + nombre + ", tienes " + edad + " años. ¡Bienvenido al
sistema!");
   }
}
```

El método debe llevar tipo de retorno void, porque realiza una acción directa (imprimir en consola) y ningún otro bloque de código necesita usar un resultado.

Ejercicio 3: Parámetros vs. argumentos. Enunciado: identificar los parámetros y los argumentos de un fragmento de código y explicar la diferencia. Un parámetro es la variable declarada en la firma del método, que actúa como espacio reservado para recibir datos; un argumento es el valor real que se envía cuando se llama al método. Aplicado al ejemplo de la clase Producto:

Término	En el ejemplo	Dónde aparece
Parámetro	double porcentajeDescuento	Definición: calcularPrecioFinal(double porcentajeDescuento)
Argumento	15.0	Llamada: p1.calcularPrecioFinal(15.0)

Ejercicio 4: El comportamiento del paso por valor. Enunciado: si la variable numero del main vale 10 antes de llamar al método, ¿cuál será su valor impreso después de ejecutarlo? Respuesta: seguirá valiendo 10. Java siempre pasa los argumentos por valor, es decir, el método recibe una copia; cualquier modificación del parámetro dentro del método afecta solo esa copia y la variable original permanece intacta.

Ejercicio 5: Creación de un método con múltiples parámetros. Enunciado: agregar a la clase Calculadora el método calcularPromedio, que reciba tres notas double (n1, n2, n3), calcule el promedio y retorne el resultado.

```
public double calcularPromedio(double n1, double n2, double n3) {
   return (n1 + n2 + n3) / 3;
}
```

Ejercicio 6: Validación de datos (control de errores). Enunciado: crear el método verificarEdad que reciba un int edad; si es menor a 0 o mayor a 120, debe imprimir «Edad inválida» y retornar false; si es correcta, retornar true.

```
public boolean verificarEdad(int edad) {
   if (edad < 0 || edad > 120) {
```

```
        System.out.println("Edad inválida");
        return false;
    }
    return true;
}
```

Ejercicio 7: Completar la clase Producto. Enunciado: rellenar los espacios vacíos de una clase Producto para que calcule el valor total del inventario de un producto. Considerando los atributos stock y precioUnitario utilizados en el ejercicio 8, la clase completa queda así:

```
public class Producto {
    int stock;
    double precioUnitario;

    public double calcularValorTotal() {
        return stock * precioUnitario;
    }
}
```

Ejercicio 8: Instanciación e invocación en el Main. Enunciado: dentro del main de una clase principal, instanciar un objeto Producto llamado p, asignarle stock = 10 y precioUnitario = 25.5, llamar a calcularValorTotal(), guardar el resultado en totalInventario e imprimirlo.

```
public class Main {
    public static void main(String[] args) {
        Producto p = new Producto();
        p.stock = 10;
        p.precioUnitario = 25.5;

        double totalInventario = p.calcularValorTotal();
        System.out.println("Total del inventario: $" + totalInventario);
    }
}
```

3.9. Comparación entre método void y método con return en la clase Producto

Característica	Método void	Método con return
¿Devuelve un valor?	No	Sí
Ejemplo en la clase	mostrarDetalles()	calcularPrecioFinal(double porcentajeDescuento)
Tipo de retorno en la firma	void	double
Cómo se invoca	p1.mostrarDetalles();	double precioOferta = p1.calcularPrecioFinal(15.0);

4. Análisis de resultados

Al ejecutar la clase Main se obtiene la siguiente salida en consola:

Producto: Laptop | Precio: $800.0
Precio final con oferta: $680.0

Instrucción	Resultado	Observación
p1.mostrarDetalles();	Producto: Laptop | Precio: $800.0	Método void: realiza una acción sin devolver valor.
p1.calcularPrecioFinal(15.0);	680.0	Método con retorno: descuento = 800.0 × 0.15 = 120.0; retorna 800.0 − 120.0 y el valor se captura en precioOferta.

La ejecución permitió comprobar que un método void solo ejecuta una acción, mientras que un método con return entrega un resultado que otra parte del programa puede guardar y reutilizar. También evidenció que el parámetro porcentajeDescuento permite aplicar distintos descuentos al mismo producto sin reescribir el método, y que el return finaliza el método devolviendo el valor calculado a la clase Main.

De esta manera, el repaso de los fundamentos, el taller «Completar código», la prueba en Main y los ejercicios APE contribuyeron a consolidar el tema.

5. Conclusiones

- Se usa void cuando el método solo ejecuta una acción y ningún otro bloque necesita el resultado; se usa un tipo de dato cuando el programa principal necesita la respuesta del cálculo.
- El return no solo devuelve el valor: también detiene la ejecución del método, por lo que el código escrito después de él es inalcanzable.
- Los parámetros son los canales de entrada del método; el parámetro es el molde y el argumento es el dato real que se envía en la llamada.
- Java siempre realiza el paso por valor: los métodos trabajan con copias y no alteran las variables originales.
- La combinación de taller en parejas, socialización y competencia interactiva permitió evaluar y consolidar lo aprendido durante la semana.

6. Recomendaciones

- Practicar la elección entre void y return analizando, en cada método, si otro bloque de código necesita su resultado.
- Probar las clases Producto y Calculadora en un entorno de desarrollo, cambiando los argumentos para observar cómo varían los resultados.
- Verificar que la firma de cada método sea coherente con su cuerpo: tipo de retorno, return y parámetros.

- Incluir validaciones en los métodos que lo requieran, como el control de rangos de edad o la división entre cero.

7. Bibliografía

Oracle. (s. f.-a). Defining methods. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/methods.html

Oracle. (s. f.-b). Passing information to a method or a constructor. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/arguments.html

Oracle. (s. f.-c). Returning a value from a method. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/returnvalue.html